

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Observation Task No.	5
Student Name	B.JATIN
Roll No.	A24126552070
Date	21-09-2026

1. TITLE

Differentiation Between JavaScript Functions and Classes

2. AIM

To understand the difference between JavaScript functions and classes and demonstrate how a class creates multiple objects with common properties and methods.

3. OBJECTIVES

1. Understand functions and classes in JavaScript.
2. Create objects using a function.
3. Define a reusable class with a constructor and method.
4. Create multiple objects from the same class.
5. Access common properties and methods through different objects.

4. THEORY

A JavaScript function is a reusable block of code and can also be used to construct or return objects. A class is a blueprint for creating objects with common properties and methods. The constructor initializes object properties, while class methods define common behavior. Each object created with new has its own property values while sharing the class method definition.

5. ALGORITHM / PROCEDURE

1. Create a function named createStudent() that returns a student object.
2. Define a Student class with a constructor accepting name and course.
3. Create an introduce() method inside the class.
4. Create three Student objects using new.
5. Call the common method for each object.
6. Display selected properties to demonstrate object-specific values.

6. PROGRAM

Source file: classes_functions.js

// Task 5: JavaScript Functions and Classes

// A function can be used to create an object with common properties.

```
function createStudent(name, course) {  
  return {  
    name: name,  
    course: course,  
    introduce: function () {  
      return `I am ${this.name} and I study ${this.course}.`;  
    }  
  };  
}
```

// A class provides a reusable blueprint for creating objects.

```
class Student {  
  constructor(name, course) {  
    this.name = name;  
    this.course = course;  
  }  
  
  introduce() {  
    return `I am ${this.name} and I study ${this.course}.`;  
  }  
}
```

```
const functionStudent = createStudent("Rahul", "CSE");  
const student1 = new Student("Jatin", "CSE");  
const student2 = new Student("Priya", "ECE");  
const student3 = new Student("Arjun", "IT");
```

```
console.log("FUNCTION EXAMPLE");  
console.log(functionStudent.introduce());
```

```
console.log("\nCLASS EXAMPLE");  
console.log(student1.introduce());  
console.log(student2.introduce());  
console.log(student3.introduce());
```

```
console.log("\nCOMMON PROPERTIES AND METHODS");  
console.log("Student 1 name:", student1.name);  
console.log("Student 2 name:", student2.name);  
console.log("Student 3 course:", student3.course);
```

7. CODE / SYNTAX EXPLANATION

Code / Syntax	Explanation
function createStudent()	Creates and returns an object using a function.
class Student	Defines a reusable blueprint for Student objects.
constructor()	Initializes properties when a new object is created.
new Student()	Creates a new object from the Student class.
this.name / this.course	Stores properties belonging to the current object.
introduce()	Common method available to Student objects.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Status
TC-01	Create student1	Student object created	Pass
TC-02	Call student1.introduce()	Student 1 message displayed	Pass
TC-03	Create three class objects	Each has its own name/course	Pass
TC-04	Access student properties	Correct values displayed	Pass

9. OUTPUT

Execution output captured from the local test run.

Task 5 - Execution Output

```
FUNCTION EXAMPLE
I am Rahul and I study CSE.

CLASS EXAMPLE
I am Jatin and I study CSE.
I am Priya and I study ECE.
I am Arjun and I study IT.

COMMON PROPERTIES AND METHODS
Student 1 name: Jatin
Student 2 name: Priya
Student 3 course: IT
```

Output summary:

FUNCTION EXAMPLE
I am Rahul and I study CSE.

CLASS EXAMPLE
I am Jatin and I study CSE.
I am Priya and I study ECE.
I am Arjun and I study IT.

COMMON PROPERTIES AND METHODS
Student 1 name: Jatin

10. RESULT

Thus, JavaScript functions and classes were differentiated, and multiple Student objects were successfully created from a common class with shared methods and individual properties.